ENME 441

Stepper Motors

Topics

PART 1

motor types

- DC, servo, stepper

PART 2

stepper overview

- VR, permanent magnet, hybrid
- Unipolar vs bipolar
- Wave, full step, half step

PART 3

stepper drivers

- Why we can’t drive by GPIO pins directly
- Transistors and flyback diodes
- Darlington arrays (ULN2003), H-bridges

PART 4

28BYJ-48 hardware and software

- Operating details
- Wiring the ULN2003 board
- Stepper driver code development

01

Motor Types

DC, servo, stepper



Main Motor Types

Brushed DC motor

"spin this fast"

- Apply a voltage and it spins
- Speed follows voltage, torque follows current
- Open loop: knows nothing about current angle, so encoders are needed

Servo motor

"go to this angle"

- DC motor + gearbox + pot + controller
- Closed loop: send an encoded angle signal; it closes the loop internally
- Limited to 180° travel (usually)

Stepper motor

"advance this many steps"

- Rotation quantized by the pole geometry
- Count the steps and you know the angle
- Open loop: counting steps can lead to errors, so encoders still required

1. Brushed DC Motor

Refer to your ENME351 notes for details on DC motor physics, torque-speed curves, driver circuits, etc.

1

Two wires, DC voltage control

Apply a voltage across the terminals and it turns (current → magnetic field → applied torque on permanent magnet of rotor). Reverse the voltage and it turns the other way. There is nothing else to command.

2

PWM sets the speed

Duty cycle maps to speed (roughly, and only under a constant load).

3

Torque ←→ current

A stalled motor draws its full stall current and produces heat instead of motion. This is where H-bridges and motors die.

4

Add an encoder for position control

Position, and even reliable speed, means measuring the shaft angle and closing the loop in software. This is a control system challenge.

WHERE TO USE

Fans, pumps, wheels, drills — anything that needs to spin at a defined speed, without regard to precise positioning.

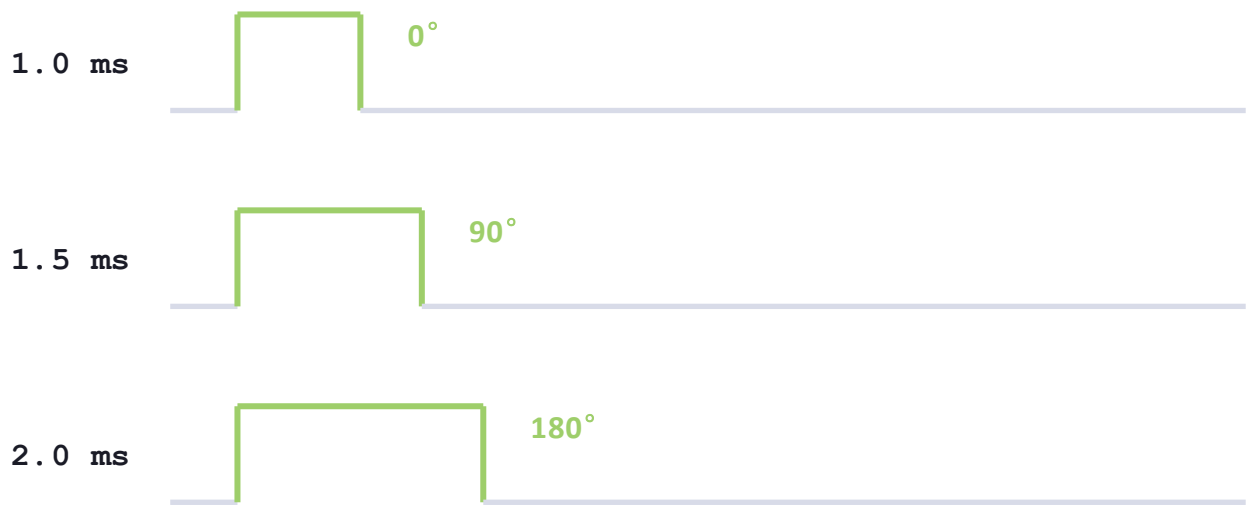
Use an H-bridge to reverse direction

One transistor can only switch a motor on and off. Reversing means swapping polarity. An H-bridge circuit uses 4 transistors, arranged in an H with the motor at the center, allowing the current path to be reversed.

2. Servo Motors

A DC motor combined with a gearbox, position sensor (potentiometer typically), and controller. Talk to the controller with angle commands encoded in a PWM signal.

Input command is the width of one pulse, repeated every 20 ms



The pulse repeats at 50 Hz. The pulse width defines the output angle.

- **The loop is closed inside the case**

  A potentiometer on the output shaft tells the built-in controller the shaft position. A control loop drives the motor until the error is zero.

- **One PWM pin to control**

  PWM(Pin(n), freq=50) and a duty value is the entire interface. The servo has its own driver, so no transistor is needed for the signal.

- **Limited travel**

  Most servos cover about 180°. Commanding past the mechanical stop can draw significant current and create jitter in the output.

3. Stepper Motors

Steppers provide rotation in fixed increments. The step angle is built into the motor geometry, and your code determines the number of steps, step direction, and step speed.

1

Position without a sensor

Every step moves the shaft by the same fixed angle. Count the steps and you know the angle, with no encoder needed (ideally).

2

The rotor holds position when stopped

Leave the coils energized and the rotor is actively held in place by magnetic force/torque. A DC motor at rest resists nothing at all.

3

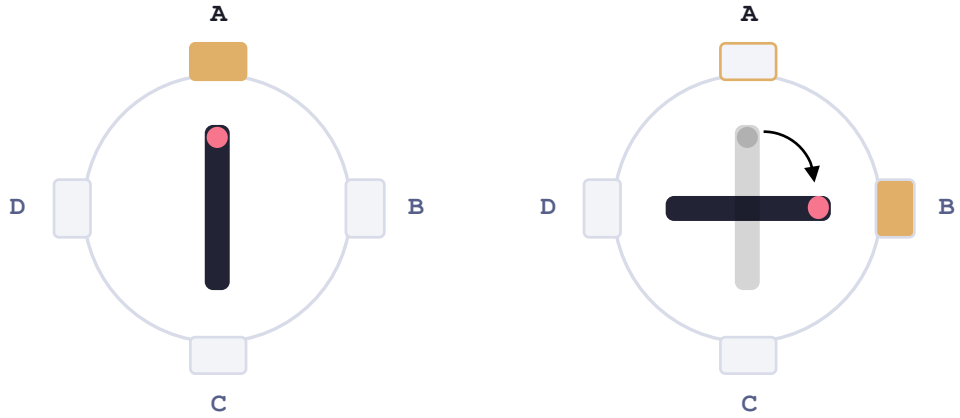
You control the commutation

There are no brushes deciding which coil is next. Your code energizes the coils in the desired sequence.

4

Open loop – use an encoder for optimal accuracy

If the load is too heavy or you step too fast, the rotor skips and the motor says nothing. Your count and the real shaft angle quietly diverge.



the rotor lines up with the energized phase

Basic idea

Energize a coil and the rotor aligns to it. Energize the next coil and the rotor re-aligns to it. Repeat for continuous rotation.

Motor Comparison

	BRUSHED DC	SERVO	STEPPER
Command input	Shaft speed	Absolute shaft angle	Change in shaft angle
Control signal	DC voltage or PWM duty cycle	1–2 ms pulse, repeated at 50 Hz	A sequence of coil patterns
Position feedback	None. Add an encoder yourself	Built in controller	None, but step count provides position
Range of motion	Continuous, unlimited	About 180°, set by the pot	Continuous, in fixed increments
Torque at standstill	Zero	Held by the internal controller	Full holding torque while energized
Pins and hardware	H-bridge, 2–3 GPIO	One GPIO with PWM	4 switched lines to a driver IC

02

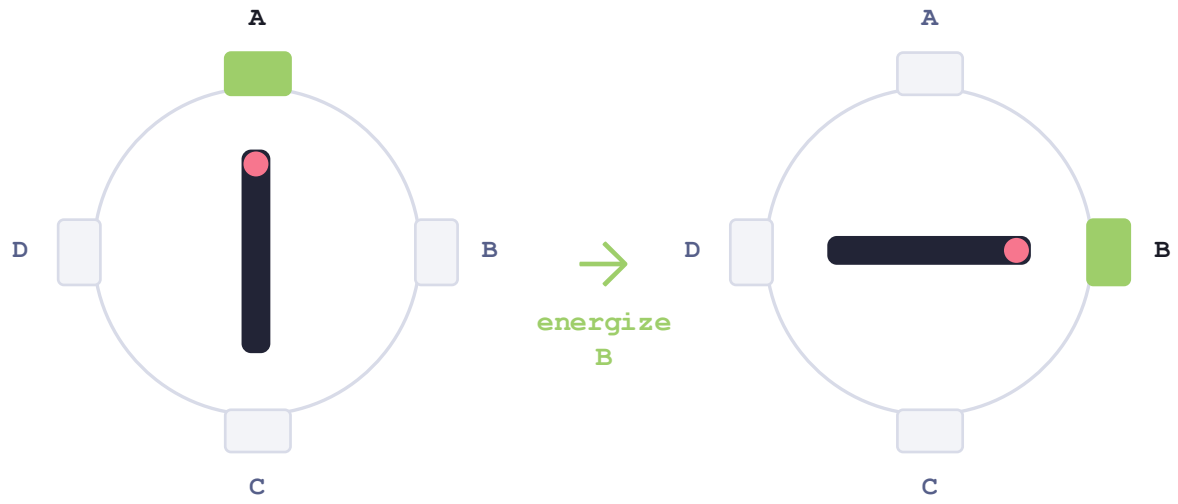
Stepper Motors

Stepper motor overview



Inside a Stepper

A permanent magnet rotor lines up with whichever pole is energized.



Pole A energized · rotor at 0°

Pole B energized · rotor at 90°

This cartoon is greatly over-simplified!

The image shows 4 poles on the stator and 1 pole on the rotor, giving a 90° step. A real motor reduces the step with a toothed rotor and stator, giving many more poles for each component.

The 28BYJ-48 yields 11.25° from the same 4 phases shown here.

- **Discrete phases**

  4 coils (4 phases) are wound in pairs on opposite poles of the stator. Energizing one phase aligns the rotor to that pole's axis.

- **Step angle is fixed**

  Determined by the motor geometry – number of stator teeth and rotor teeth

- **Detent torque with the power off**

  A permanent-magnet rotor is still magnetized when nothing is energized, so the shaft has faint preferred positions. You can feel them by hand.

Stepper Motors Types

Permanent magnet

permanent magnet rotor

- The rotor is a magnetized cylinder with alternating north and south sections
- Attraction and repulsion both do work, so **torque per amp is high**
- Step angles are relatively coarse: 7.5°, 11.25°, 15° are typical
- Common for small and inexpensive steppers (e.g. the common 28BYJ-48).

Variable reluctance

toothed iron rotor, no magnet

- The rotor is soft iron with teeth and carries no magnetism of its own
- It rotates to the position of least magnetic reluctance (aligned with energized poles)
- **Fine step angles** are possible, but **torque is low** and there is no detent torque at all when unpowered
- Rare in small products

Hybrid

toothed and magnetized

- A toothed rotor with an axial permanent magnet leverages both effects at once
- **Fine step angles and high torque**: 1.8° per step, 200 steps per revolution, is standard
- Common example: NEMA-17 (found in 3D printers and small CNC machines)
- More expensive than permanent magnet steppers

Three Ways to Energize the Coils

Wave drive

one phase on

	IN1	IN2	IN3	IN4	
0b0001	1	0	0	0	A
0b0010	0	1	0	0	B
0b0100	0	0	1	0	C
0b1000	0	0	0	1	D

Full step

two phases on

	IN1	IN2	IN3	IN4	
0b0011	1	1	0	0	A + B
0b0110	0	1	1	0	B + C
0b1100	0	0	1	1	C + D
0b1001	1	0	0	1	A + D

Half step

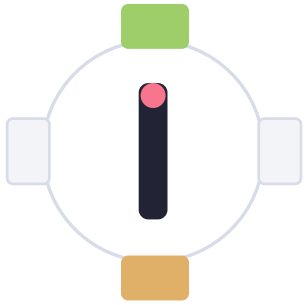
alternating

	IN1	IN2	IN3	IN4	
0b0001	1	0	0	0	A
0b0011	1	1	0	0	A + B
0b0010	0	1	0	0	B
0b0110	0	1	1	0	B + C
0b0100	0	0	1	0	C
0b1100	0	0	1	1	C + D
0b1000	0	0	0	1	D
0b1001	1	0	0	1	A + D

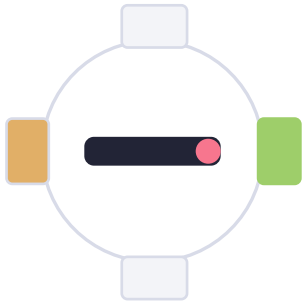
Wave drive requires only a single direction for stator coil current, but pulls ~70% of the torque of full step drive (at half the total current).
Full step drive doubles stepping resolution over wave drive.
Half step drive combines both methods, alternating every other step.

Wave drive cycle

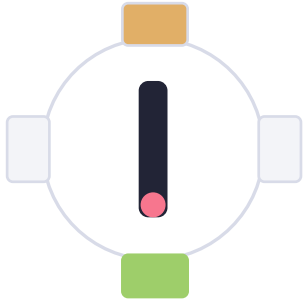




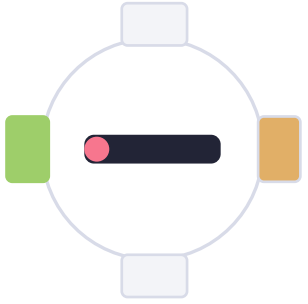
step 1 · A



step 2 · B



step 3 · C



step 4 · D

In a real motor the toothed geometry scales the same 8 states down to a smaller angle (5.625 ° for the 28BYJ-48)

Full Step Cycle





step 1 · A + B



step 2 · B + C



step 3 · C + D

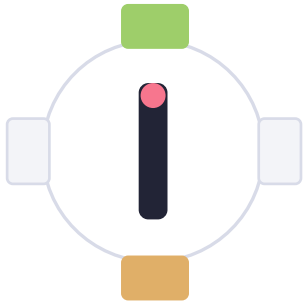


step 4 · A + D

In a real motor the toothed geometry scales the same 8 states down to a smaller angle (11.25 ° for the 28BYJ-48)

Half Step Cycle

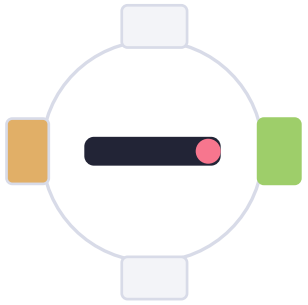




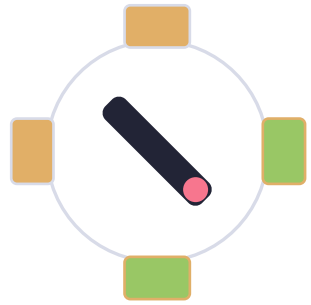
step 1 · A



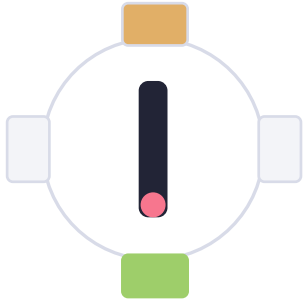
step 2 · A + B



step 3 · B



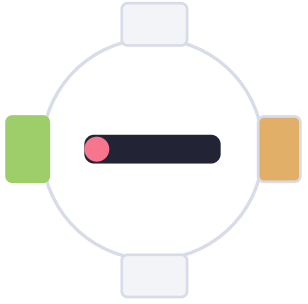
step 4 · B + C



step 5 · C



step 6 · C + D



step 7 · D



step 8 · A + D

In a real motor the toothed geometry scales the same 8 states down to a smaller angle (5.625 ° for the 28BYJ-48)

Unipolar vs. Bipolar Stators

Unipolar vs. bipolar stators require different drive circuits to reverse current (as needed for full step and half step drive).

Unipolar

5 or 6 wires · center-tapped

Each phase winding has a tap at its middle, wired to the supply. To energize one half the winding, pull its end to ground.

- Current never reverses, so can use a simple low-side switch
- 4 switches drive 4 half-windings
- Only half the copper is carrying current at any moment, so torque per unit size is lower
- The 28BYJ-48 is unipolar (center taps joined to the single red wire)

Bipolar

4 wires · no center taps

Two plain windings with no middle connection. To reverse the field you have to reverse the current through the whole coil.

- Needs an H-bridge per winding — 4 switches each, 8 in total
- The entire winding is energized every step, so more torque from the same motor
- The NEMA-17 is bipolar
- A 6-wire unipolar motor can be run as bipolar: ignore the center taps

Half Step Tradeoffs

1 **Torque alternates step to step**

Single-coil wave drive produces roughly 70% of the torque of two-coil full drive. Half-stepping strings them together, so every other step is a weaker one.

2 **Current alternates too**

Two coils on draws twice the current of one. The supply sees a pulsing load, and the average current sits between the wave and full-step figures.

3 **Smoother, and quieter**

Halving the jump size halves the impulse the rotor and the load absorb at each step. Half-stepping visibly reduces the ringing you can hear and feel.

4 **Beyond half step drive: microstepping**

Drive the two coils with unequal currents instead of just on and off, and the rotor sits anywhere between the poles. Requires a driver that can regulate current.

STEPS PER REVOLUTION, 28BYJ-48

Wave drive

2048 steps — 0.176° each

lowest torque, lowest current

Full step

2048 steps — 0.176° each

about 1.4× the torque, 2× the current

Half step

4096 steps — 0.088° each

twice the resolution, uneven torque

Counts are at the output shaft, through the 64:1 gearbox.

03

Stepper drivers

Circuits to switch the stepper coils



Why Not Drive Straight From a GPIO Pin?

Current

One 28BYJ-48 phase is about 50 Ω. At 5 V that is 100 mA, and two phases are energized on half of the steps → 200 mA. An ESP32-C3 pin is limited to tens of milliamps range as an absolute maximum.

Voltage

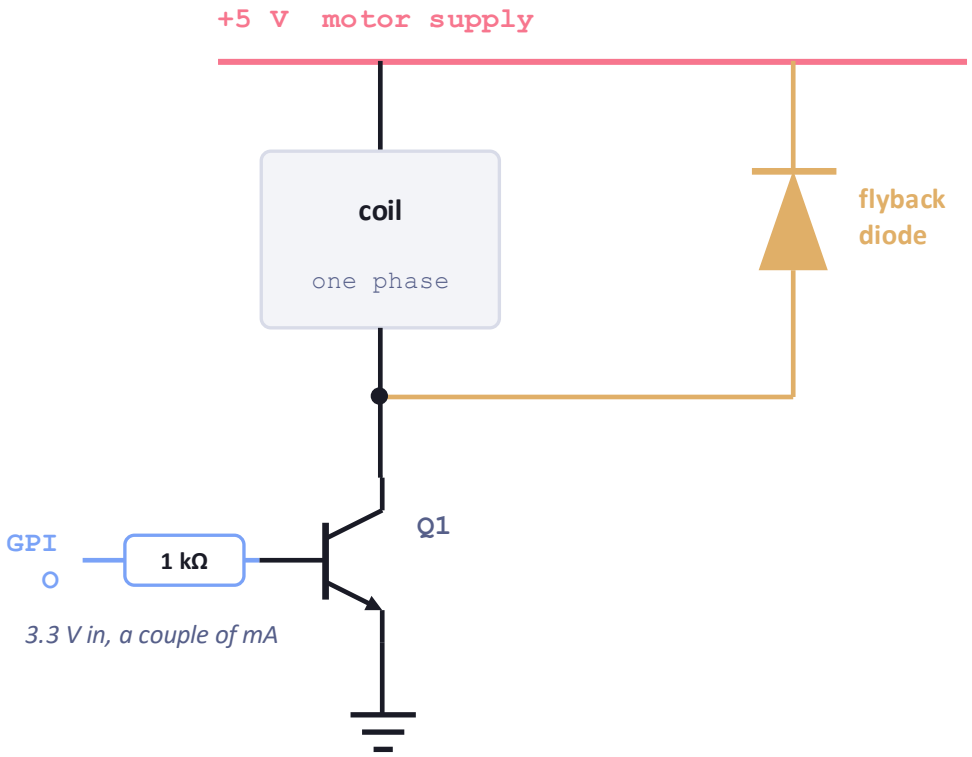
The motor wants 5 V across the coil. A C3 output pin swings to 3.3 V, so even if the current were ok you would be running the motor on two-thirds of its rated voltage, (yielding under two-thirds of its torque).

Inductive kickback

A coil resists a change in current: $V = -L \cdot di/dt$. Switch off 100 mA in a microsecond and the coil generates tens of volts of the opposite polarity, straight back into whatever opened the circuit. That is how GPIO pins die!

Bipolar Transistor + Kickback Diode

The minimum viable driver for a unipolar phase: a transistor to ground, and a diode to shunt kickback current.



Repeat four times (one per phase) and you have a full unipolar stepper driver.

- **The transistor is a low-side switch**

  The coil sits between +5 V and the collector. Driving the base high pulls the collector to near ground and current flows. The motor supply never touches the C3.

- **The diode is not optional**

  Reverse-biased and idle while the coil is energized. The instant the transistor turns off, it becomes the only path the collapsing coil current can take, clamping the spike to about 0.7 V above the rail.

- **Base resistor sets the base current**

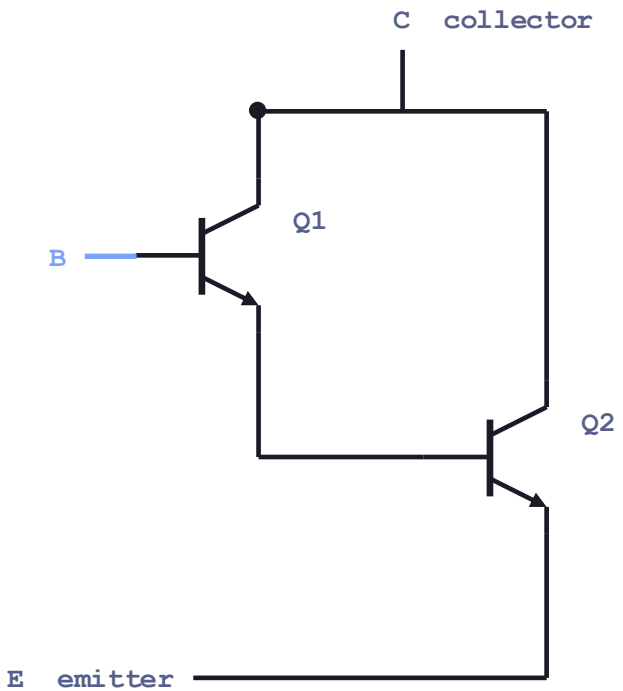
  Roughly (3.3 − 0.7)/1 kΩ ≈ 2.6 mA. With a gain of 100 that supports well over 200 mA of collector current, with margin to stay saturated.

- **Lots of parts needed!**

  4 transistors, 4 resistors, 4 diodes, and 13 connections per motor…there is a chip for this!

The Darlington Pair

Two BJTs wired with output current from the first serving as the second's input current, thereby multiplying the overall current gain



Current Gain

$$\beta_{total} \approx \beta_1 \times \beta_2$$

100 × 100 = 10 000. A milliamp into the base of BJT_1 controls an amp at the collector of BJT_2.

Why not a MOSFET?

For most switching it is the better modern answer: milliohms of on-resistance instead of a volt of drop, and no base current at all. Darlingtons survive because you can buy seven of them, with their diodes, for pennies.

Limitations:

- **Two V_{BE} drops in series**

  It takes about 1.4 V at the base to turn it on, not 0.7 V. Comfortable from 3.3 V logic, but it rules out lower-voltage signalling.

- **It never fully saturates**

  Q1's collector cannot fall below Q2's base, so the on-state drop is about 0.9–1.2 V instead of 0.2 V. That is heat in the package, and voltage subtracted from your coil.

- **Slow to turn off**

  Charge stored in Q2's base has nowhere convenient to go. Irrelevant at a millisecond per step; it matters at tens of kilohertz.

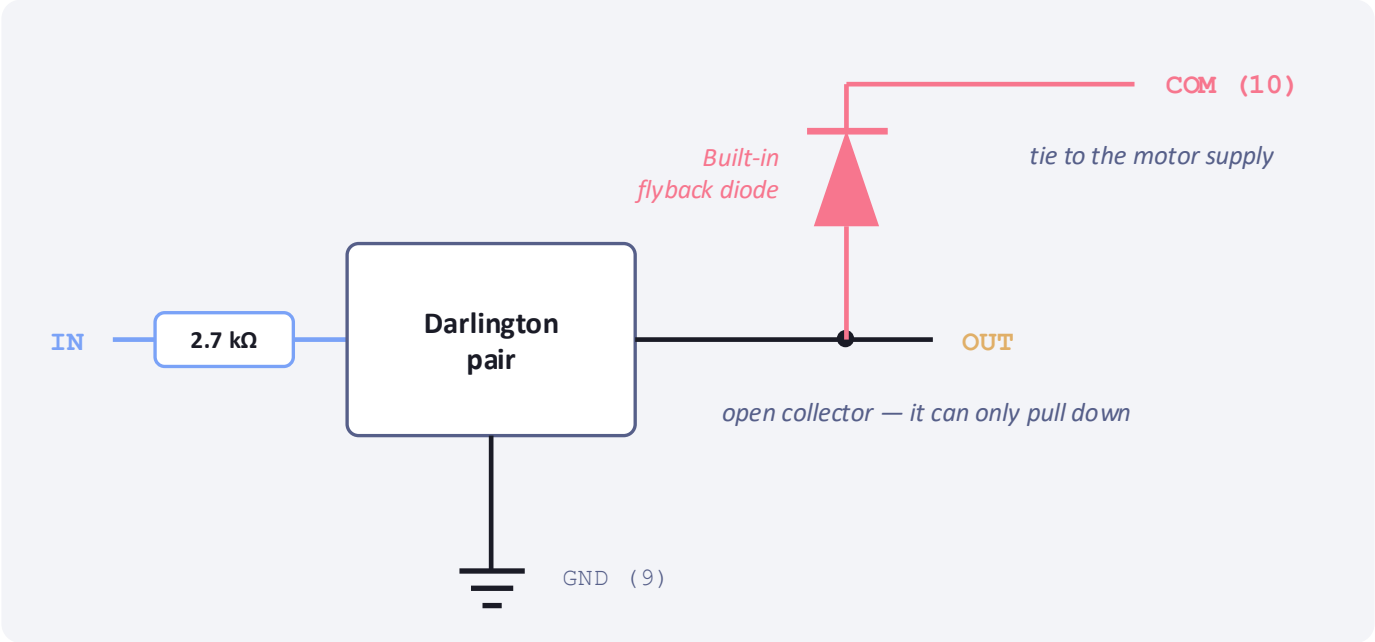
- **It only sinks**

  This is an NPN pair: it pulls its output down to near ground. The load has to sit between the supply and the output (which is exactly how a unipolar coil is wired).

ULN2803: 8x Darlington Pair Array IC

For a single motor (4 outputs), a **ULN2003** with 7 outputs is commonly used.
To drive 2 stepper motors (8 outputs) we will instead use the **ULN2803** with 8 high-current outputs.

ONE CHANNEL, INSIDE THE PACKAGE



Drive IN high and OUT sinks to ground, energizing the coil hung between it and +5 V.

- **ULN2803 has 8 channels**

  Inputs 1B–8B on pins 1–8, outputs 1C–8C on pins 18–11. A four-phase stepper uses four of the eight and leaves four spare.

- **Integrated 2.7 kΩ input resistor**

  It sets base current from a logic-level input, so you wire a 3.3 V GPIO straight to the pin. No external resistor, no level shifter.

- **500 mA per channel, 50 V standoff**

  Far beyond a 28BYJ-48. The real limit is heat dissipation since driving all 8 circuits hard at once can overheat the chip!

- **Pin 10 is a common cathode**

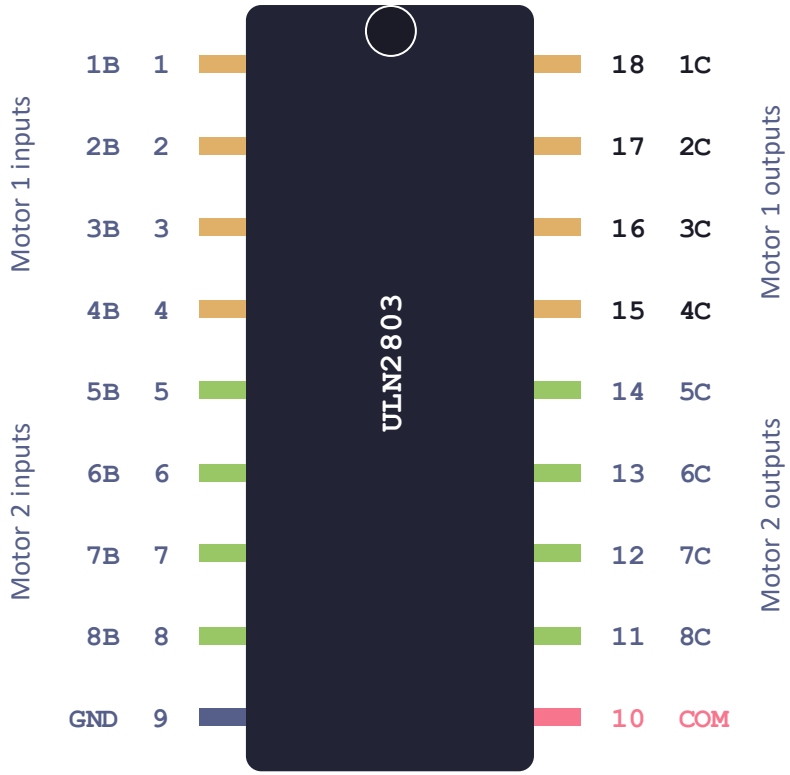
  Pin 10 is a common cathode for all 8 internal **flyback diodes**. Tie it to the motor supply and every coil gets its current shunt for nothing.

- **Inverted logic**

  Input high → output low → current flows

Wiring the ULN2803 to a Unipolar Stepper

Inputs down one side, outputs down the other



SIGNAL	GOES TO	WHY
1B – 4B	GPIO 0, 1, 3, 4	one logic input per phase
1C – 4C	Motor coil wires	each sinks its phase to ground
COM (10)	+5 V motor supply	cathode of the internal flyback diodes
GND (9)	GND, shared with the C3	the logic needs a common reference

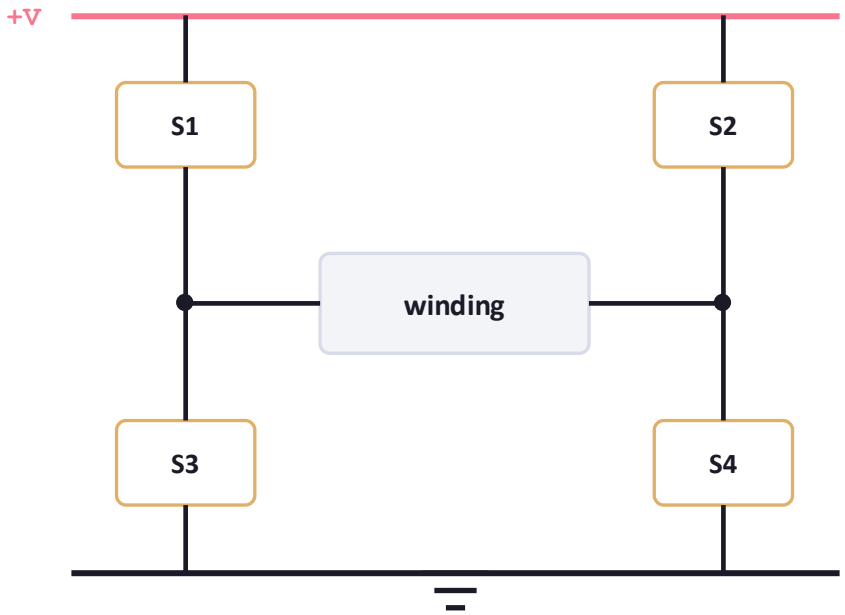
There is no VCC pin

The ULN2803 has no logic supply of its own — only GND and COM. It is not powered so much as it is referenced: the inputs are just base drive, and COM only exists to give the diodes somewhere to clamp to.

OPTIONAL

H-Bridge for Bipolar Drive Steppers

A Darlington array can only pull an output down (sufficient for reversing current in unipolar motors). Reversing current through a bipolar coil needs four switches.



S1 + S4 on → current flows left to right. S2 + S3 on → it flows right to left. Never S1 + S3, or S2 + S4 — that shorts the supply straight to ground.

- **A bipolar motor needs two of these**

  One H-bridge per winding, so eight switches. That is the price of putting current through the whole coil instead of half of it.

- **L293D H-bridge**

  Provides 4 half-bridges in a single IC with 2 enable pins and clamp diodes on chip. It handles 600 mA per channel.

- **Shoot-through is a failure mode**

  Turning on both switches in one leg puts the supply directly across two transistors. Dedicated stepper motor driver ICs interlock the inputs to prevent this.

- **Operation sequence is more complex**

  Half-stepping a bipolar motor is the same eight rotor states, but each one now has to be expressed as a direction for each of two windings, not four on/off lines.

OPTIONAL

Dedicated Stepper Motor Driver ICs

Many good options: A4988, DRV8825, TMC2209, etc. More expensive ($5-10+) and generally for bipolar drive motors only

1

STEP and DIR commands simplify operation

One rising edge on STEP advances the motor by one microstep, in whichever direction DIR says. The chip owns the commutation table entirely.

2

Current chopping instead of resistance

The driver measures coil current and switches the supply on and off to hold it at a set limit. That lets you run a 2 V motor from 24 V and get the current up fast (for high-speed torque)

3

Microstepping is built in

Once current is regulated, you can hold two windings at unequal fractions of full current and park the rotor between poles. 1/16 and 1/32 steps are routine.

4

Robust protection

Thermal shutdown, overcurrent detection, and a sleep pin typical

THE WHOLE INTERFACE

```
dir.value(1)
for i in range(200):
    step.value(1)
    time.sleep_us(2)
    step.value(0)
    time.sleep_us(800)
```

Choosing a Driver

Three circuits, three different divisions of labour between your code and the hardware.

	DARLINGTON ARRAY	H-BRIDGE	STEP / DIR DRIVER
Typical part	ULN2003	L293D, DRV8833	A4988, DRV8825, TMC2209
Motor type	Unipolar only	Bipolar	Bipolar
GPIO pins	4 — one per phase	4 inputs, plus enables	2 (STEP and DIR)
Who writes the sequence	Your code, every step	Your code, every step	The chip. You just pulse STEP
Current control	None — the coil's resistance sets it	None	Chopper, with an adjustable limit
Microstepping	No. Full and half step only	No	Down to 1/16 or 1/32
Flyback diodes	Built in, via the COM pin	Built in on the L293D	Built in

04

28BYJ-48 hardware and software



28BYJ-48 Stepper Motor

A unipolar stepper that is relatively slow and weak but remarkably precise.

Type	4-phase unipolar PM	Wires	5 (4 phases + common)
Supply	5 V DC (12 V parts exist)	Coil resistance	≈ 50 Ω per phase
Current	≈ 100 mA per phase	Internal step angle	11.25° / 5.625° half
Gear reduction	nominally 64:1	Output resolution	4096 half-steps / rev
Holding torque	≈ 34 mN·m (350 g·cm)	Practical top speed	10–15 rpm at the shaft



Why so slow?

A 64:1 gearbox trades speed for torque and resolution.

4096 Half Steps per Shaft Revolution

32

full steps per turn
of the internal rotor

× 2

half-stepping doubles
the number of states

× 64

gearbox between
rotor and output shaft

= 4096

half-steps per turn of
the output shaft

360° / 4096 = 0.0879° per half-step at the output shaft

64:1 is a rounded number

The real gear train is 32/9 × 22/11 × 26/9 × 31/10 = 63.68395 : 1, so a revolution is really 4075.77 half-steps, not 4096. Using 4096 overshoots by about 0.5% — roughly 1.8° of extra rotation every turn, and it accumulates.

For precise positioning, use 4075.77 and round the step count. Better still, re-home against a limit switch.

Wiring a Single 28BYJ-48 Motor to a ULN2803 Driver

28BYJ-48	ULN2803	NOTES
orange (A)	pin 18 (1C)	driven by input pin 1 (1B)
yellow (B)	pin 17 (2C)	driven by input pin 2 (2B
pink (C)	pin 16 (3C)	driven by input pin 3 (3B)
blue (D)	pin 15 (4C)	driven by input pin 4 (4B)
red	+5 V	common center tap

Where the 5 V comes from

The SuperMini's 5 V pin is fed from USB. A 28BYJ-48 pulling 200 mA is within USB spec. If the board browns out and resets mid-rotation, move the motor to its own supply and keep the grounds tied together.

A stationary motor still uses power

A stepper draws full current at zero rpm. If you do not need holding torque, write zeros to all four pins when you stop.

stepper.py

Basic bit-banged stepper driver code using a ULN2003 or ULN2803 Darlington array

```
# stepper.py
#
# Half-stepping a 28BYJ-48 through a ULN2003
#
# ESP32-C3 SuperMini

from machine import Pin
import time

# Controller inputs: in1, in2, in3, in4
# GPIO 2 is skipped — it is a boot strap pin
pin_nums = [0, 1, 3, 4]
pins = [Pin(n, Pin.OUT) for n in pin_nums]

delay = 1200        # between half-steps [us]

# Half-step sequence for CW motion:
seq = [0b0001,      # A
       0b0011,      # A + B
       0b0010,      # B
       0b0110,      # B + C
       0b0100,      # C
       0b1100,      # C + D
       0b1000,      # D
       0b1001]      # D + A
```

- **The seq list contains the 4-bit half step inputs**

  Each bit corresponds to one phase:
  0 = phase is off
  1 = phase is on

- **1200 µs is a working value, not a hard limit**

  Fast enough to be useful, slow enough that the coil current approaches its full value before the next state. Below ~1000 µs the motor will start skipping under load.

OUTPUT SHAFT SPEED

current rise time	~1200 µs
per revolution	4096 × 1.2 ms ≈ 4.9 s
output speed	~12 rpm

Execute the Steps

3 nested loops: Revolutions on the outside, the eight-state half step cycle in the middle, the four pins on the inside.

```
def loop(dir):          # dir = 1 (cw) or -1 (ccw)
    pos = 0             # position in the sequence
    # 512 cycles per output revolution
    # (8 cycles per rotor turn x 64 gear ratio):
    for i in range(512):
        for halfstep in range(8):        # 8 states per cycle
            for j in range(4):           # walk the 4 pins
                pins[j].value(seq[pos] & (1<<j))
            pos += dir       # next position in the sequence
            pos %= 8         # stay in [0, 7]
            time.sleep_us(delay)

try:
    loop(1)
    loop(-1)
except Exception as e:
    print(e)
```

512 × 8 = 4096 half-steps per turn of the output shaft. loop(1) then loop(-1) sends it one revolution clockwise and one back.

- **seq[pos] & (1 << j) tests bit j**

  The result is 0 or 2^j (Pin.value() takes any non-zero as HIGH)

- **pos += dir, then pos %= 8**

  Adding −1 and taking the modulus walks the ring backwards: 0 becomes −1, and −1 % 8 is 7 in Python. Direction really is nothing more than which way you step through the table.

- **The pins do not change together**

  The four value() calls happen in sequence, microseconds apart. The coils do not care at this timescale, but it is worth knowing that no step is truly atomic here.

- **Nothing de-energizes the coils**

  When loop() returns, the last pattern is still driving current through one or two windings. The motor holds position and gets warm. Add a final all-zeros write if you want to return to detent torque under zero power.

ESP32 → Shift Register → Darlington Array

Use a shift register to increase motor count without requiring more GPIO pins

```
# stepper_with_shifter.py
#
# 28BYJ-48 via a 74HC595 into a ULN2003.
# Four coil lines, zero extra GPIO pins.

from machine import Pin
from shifter import Shifter
import time

s = Shifter(data=5, latch=6, clock=7)

seq = [0b0001, 0b0011, 0b0010, 0b0110,
       0b0100, 0b1100, 0b1000, 0b1001]

delay = 1200            # between half-steps [us]
STEPS_PER_REV = 4096    # half-steps per output turn

def rotate(turns, dir=1):
    pos = 0
    for i in range(int(turns * STEPS_PER_REV)):
        s.shiftByte(seq[pos])
        pos = (pos + dir) % 8
        time.sleep_us(delay)
    s.shiftByte(0)      # release the coils
```

- **One byte replaces four pins**

  shiftByte() sends the same 4-bit pattern the direct version wrote pin by pin. The '595 outputs drive the ULN2003 inputs.

- **The upper nibble is a second motor**

  seq[p2] << 4 | seq[p1] runs two steppers off one byte, and off the same three GPIO pins.

- **Eight clock pulses per step**

  Bit-banging a byte through Shifter costs about a millisecond, which is the same order as the step delay itself. If you need speed, drive the '595 over SPI instead.

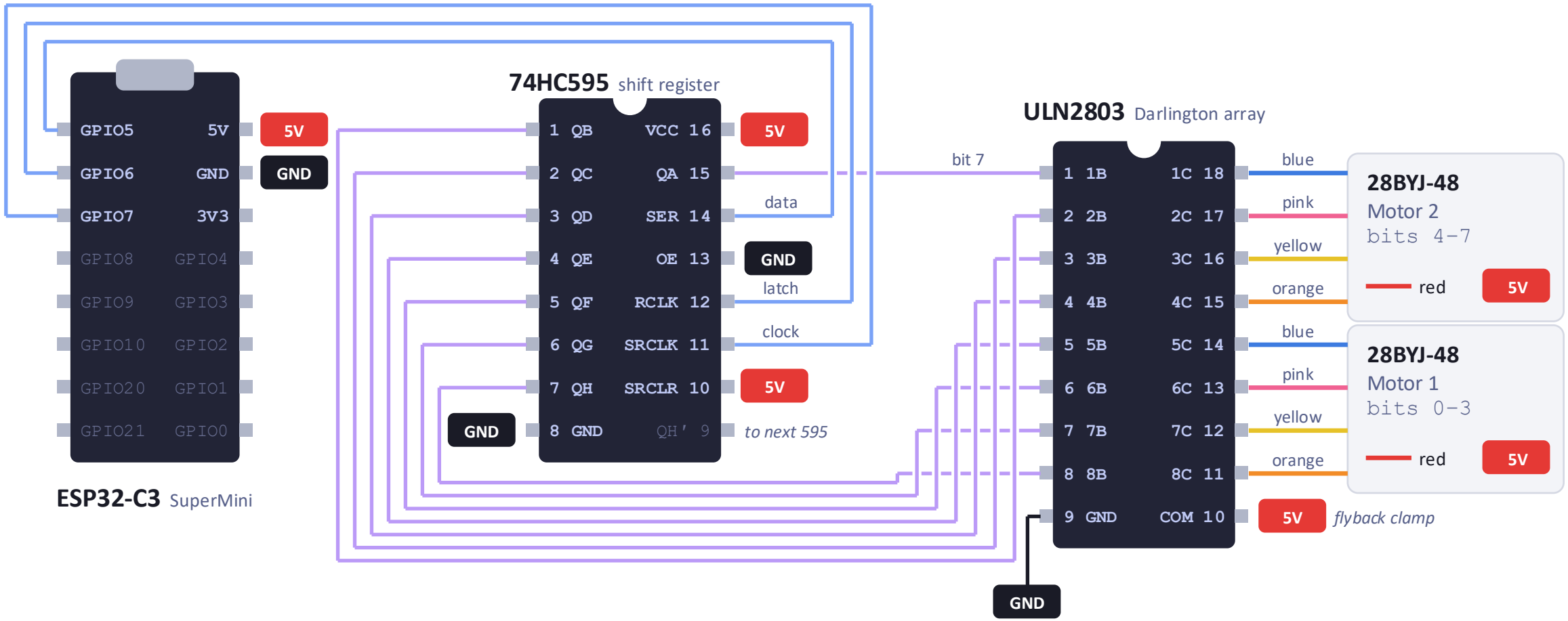
Wiring Diagram (2 motors)

Test your hardware and code

Wire up a single motor to your ESP32-C3, ‘595, and ULN2308, and run the provided *stepper_async_timer_shifter.py* code file.

No socket on a bare ULN2803

Push male jumper wires into the motor's 5-pin plug. The plug order is not always the phase order, so wire by color (blue, pink, yellow, orange, red), not by position.



Troubleshooting

Motor hums and shakes but does not turn

Two phase lines are transposed, so the sequence is no longer walking adjacent poles. Swap the middle pair of the four and try again.

Motor turns the wrong way

Step the table backwards instead of rewiring anything. Direction is only the order you walk the eight states.

The board resets partway through

The motor is browning out the 5 V rail. Give it its own supply, keep the grounds tied, and check the current your USB port will actually deliver.

Motor gets hot when it is not moving

Normal. Full current flows through the last energized coils forever. Write zeros to the pins when you are done stepping.

Motor loses count over many turns

Either you are skipping steps, or you are using 4096 when the real figure is 4075.77. Slow it down first, then fix the arithmetic.

Circuit works with LEDs but not the motor

The ULN2003's own LEDs prove your sequence and your GPIO wiring. If those chase correctly, the fault is downstream: supply, plug, or the motor itself.

Lab 8 – Stepper Motor Control



Next Steps

PCB design using KiCad

sleep_us() in a loop blocks everything else. A stepper that has to run while a web server answers requests is the concurrency lecture's problem, restated in hardware.